TECHNICAL REPORT & ARCHITECTURE WHITEPAPER

Cloud-Native Network Architecture for Defense-in-Depth of Enterprise AI Services

Experimental Mitigation of DDoS Attacks and Semantic Resource Exhaustion

Author:

Davide Torelli

https://github.com/opsdavide

Document Status: Public Release (v1.0)
Tech Stack: Nginx, FastAPI, Redis (Lua), JWT, Docker
Domain: Cloud Security, Network Security, LLM Infrastructure Protection
Code & Docs License: MIT License

October 2026

Abstract

The integration of Large Language Models (LLMs) into cloud-native services introduces an asymmetric attack surface: ultra-low-bandwidth network payloads can trigger critical hardware resource exhaustion (GPU VRAM and compute threads). This project designs, implements, and experimentally validates a microservice-based defensive architecture grounded in the *Defense-in-Depth* principle and the *Zero Trust* paradigm. The framework combines: (1) an Nginx Reverse Proxy for Layer 4/7 volumetric mitigation, (2) an asynchronous API Gateway (FastAPI) featuring cryptographic Role-Based Access Control (JWT) and dynamic rate limiting powered by an atomic *Sliding Window* algorithm (Redis and Lua scripting), and (3) an application-layer IDS module (*Payload Inspector*) designed to detect malicious or recursive prompts. Load testing conducted across standardized suites of 100 concurrent requests ($N = 10$) demonstrates a 95% reduction in abusive traffic reaching the backend inference server and stabilizes response latencies under semantic DoS attacks.

Contents

1 Introduction, Threat Modeling, and Security Objectives **1**
- 1.1 The Enterprise Cloud AI Application Landscape . . . 1
- 1.2 Threat Modeling: Semantic Denial of Service and Resource Exhaustion . . . 1
- 1.3 Project Objectives and Defense-in-Depth Paradigm . . . 2

2 System Fundamentals, Networking, and VPC Topology **4**
- 2.1 Execution Environment and Virtualization: WSL 2 and Docker . . . 4
- 2.2 Networking Fundamentals and TCP Protocol Security . . . 4
  - 2.2.1 Port Mapping and Topological Decoupling . . . 4
  - 2.2.2 Mitigating IP Spoofing via the TCP 3-Way Handshake . . . 5
- 2.3 Multi-Tier Network Topology (VPC) . . . 6
- 2.4 Microservice Composition and Docker Compose Configuration . . . 7
  - 2.4.1 Orchestration Specification: docker-compose.yml . . . 8

3 Implementation of the Three Defense-in-Depth Layers **10**
- 3.1 Layer 1: Perimeter Volumetric Filtering (Nginx) . . . 10
  - 3.1.1 Technical Breakdown of Nginx Parameters . . . 11
- 3.2 Layer 2: Role-Based Access Control (RBAC) and Atomic Rate Limiting . . . 11
  - 3.2.1 JSON Web Token (JWT) Authentication Model . . . 11
  - 3.2.2 Mitigating Race Conditions (TOCTOU) and Shared State via Redis . . . 12
  - 3.2.3 Cryptographic Audit Logging with HMAC-SHA256 Signatures . . . 13
- 3.3 Layer 3: Application IDS and Payload Inspector . . . 13

4 Experimental Validation and Results Analysis **15**
- 4.1 Measurement Methodology and Testbed Setup . . . 15
  - 4.1.1 Ingress Points and Methodological Layer Decoupling . . . 15
- 4.2 Computational Load Modeling and Telemetry . . . 16
- 4.3 Comparative Analysis of Experimental Scenarios . . . 18
- 4.4 Time Efficiency and Drop Latency Analysis . . . 18

5 Design Trade-offs, Limitations, and Future Work **20**
- 5.1 Critical Analysis of Architectural Trade-offs . . . 20
- 5.2 Stated Study Limitations . . . 20
- 5.3 Future Work . . . 21
- 5.4 Conclusions . . . 21

Bibliography **21**

Chapter 1

Introduction, Threat Modeling, and Security Objectives

This chapter defines the operational context of cloud-hosted artificial intelligence services, the specific threats associated with computational resource exhaustion (semantic DoS), and the security objectives pursued by the proposed architecture.

1.1 The Enterprise Cloud AI Application Landscape

Integrating Large Language Models (LLMs) into enterprise workflows has fundamentally transformed application service architectures. Unlike traditional software, generative artificial intelligence systems do not operate as standalone local scripts, but rather as complex, distributed cloud-native microservices exposed to internal and external clients via REST APIs.

Each inference request imposes a heavy, asymmetric computational load on backend nodes:

- **Rapid Memory Allocation**: Loading model weights and initializing the context window requires instantaneous allocation of gigabytes of dedicated GPU memory (VRAM).
- **Intensive Parallel Computation**: Tensor processing engages matrix compute cores (Tensor Cores) for the entire duration of text generation.
- **KV Cache Occupation**: Maintaining the attention state (*Key-Value Cache*) during sequential token generation locks server resources for several seconds.

As illustrated in Figure 1.1, this architecture exposes the backend to a severe cost asymmetry between the client and the server.

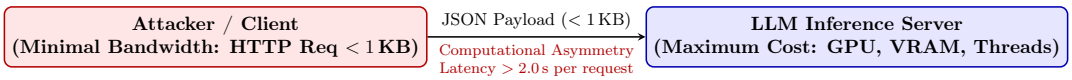


Figure 1.1: Resource asymmetry model in LLM-based application services.

1.2 Threat Modeling: Semantic Denial of Service and Resource Exhaustion

In conventional web services, a *Denial of Service* (DoS or DDoS) attack typically aims to saturate network bandwidth (Layer 3/4 volumetric attacks such as UDP Floods and SYN Floods) or exhaust the web server's connection pool (Layer 7 HTTP Floods, Slowloris).

In AI-driven services, however, a distinct and more insidious vulnerability emerges: **Semantic Resource Exhaustion Denial of Service**, identified as a primary threat in the *OWASP Top 10 for LLM Applications* (category LLM04) [2] and the *OWASP API Security Top 10* standard (API4:2023 - Unrestricted Resource Consumption) [3].

The primary attack vectors identified in the threat model are:

1. **Payload Asymmetry**: The attacker transmits a minimal JSON payload (a few hundred bytes) containing complex instructions. The backend must execute billions of floating-point operations to complete processing, stalling execution queues for legitimate requests.
2. **Volumetric Prompts**: Submitting prompts with an excessive character count that saturates the allowed context window, forcing the server to continuously reallocate tensors in memory.
3. **Generative Complexity Prompts**: Instructions crafted to force the model into producing maximum output length (*max_tokens*), prolonging GPU lock times.
4. **Lexical Repetition Prompt Flood (Spam)**: Rapid submission of prompts containing repeated identical words (low entropy), designed to saturate the tokenization pipeline without contributing semantic value.

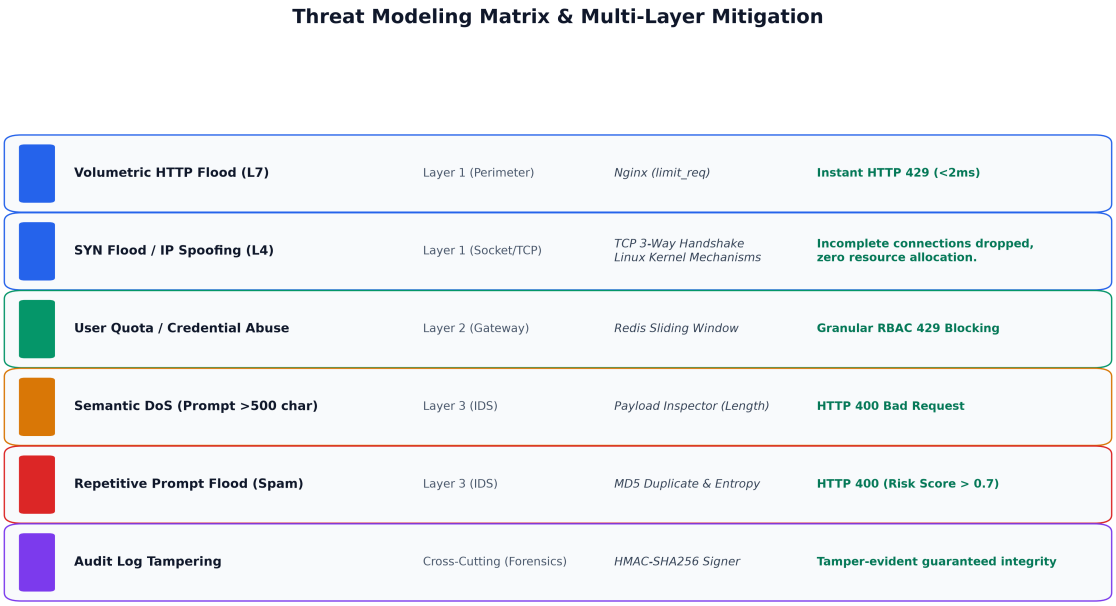


Figure 1.2: Threat matrix and corresponding multi-layer mitigation mechanisms.

As summarized in Figure 1.2, each attack vector requires a dedicated countermeasure deployed at the appropriate network layer, preventing anomalous traffic from reaching the inference server.

1.3 Project Objectives and Defense-in-Depth Paradigm

The objective of this project is to design, implement, and experimentally validate a cloud-native network architecture grounded in the **Defense-in-Depth** principle and the **Zero Trust** paradigm [1].

To ensure continuous availability of the AI service, the defense strategy is structured across three sequential, complementary layers, supported by cryptographic forensic auditing:

1. **Layer 1 - Perimeter Volumetric Protection (Nginx)**: Intercepts and mitigates raw traffic spikes via IP-based rate limiting, dropping excess requests with immediate responses (`HTTP 429`) in under 2 milliseconds.
2. **Layer 2 - Access Control and Dynamic Quotas (API Gateway + Redis)**: Authenticates clients via cryptographic JSON Web Tokens (JWT) and enforces role-based differentiated quotas (RBAC) using an atomic sliding window algorithm backed by a distributed Redis cache.
3. **Layer 3 - Heuristic Payload Inspection (Application IDS)**: Inspects prompt text by computing a risk score based on character length, lexical repetition ratio, and temporal duplication, blocking malicious prompts with an `HTTP 400 Bad Request` status code.
4. **Forensic Traceability (HMAC Audit Logging)**: Records every policy decision event in structured logs cryptographically signed via HMAC-SHA256 to guarantee log integrity and tamper resistance.

Chapter 2

System Fundamentals, Networking, and VPC Topology

This chapter analyzes the architectural foundations of the infrastructure, ranging from the virtualization and containerization environment to the isolated Virtual Private Cloud (VPC) network topology and TCP transport mechanisms. The goal is to establish a robust, reproducible systems baseline upon which to deploy the mitigation pipeline protecting the AI services.

2.1 Execution Environment and Virtualization: WSL 2 and Docker

Proper microservice segregation and compliance with production Linux environments require native kernel primitives such as *cgroups* (control groups), *namespaces*, and virtual routing tables (*iptables/nftables*).

On a Windows host platform, this stack runs via **WSL 2 (Windows Subsystem for Linux)**, which provides a full Linux environment without the overhead of a traditional heavy virtual machine:

- **Native Linux Kernel on a Type-1 Hypervisor**: WSL 2 executes an optimized Linux kernel inside a lightweight VM managed by the system hypervisor (Hyper-V). This guarantees native support for all system calls (*syscalls*) and I/O performance comparable to a bare-metal Linux installation.
- **Docker Desktop Integration**: The Docker daemon operates within the service Linux distribution (`docker-desktop`). Containers communicate directly via Unix sockets and virtual Linux network interfaces created within the WSL 2 namespace.
- **Separation Between Test Environment and Infrastructure**: Synthetic load and attack scripts (`load-test/`) reside in the host's Python virtual environment (`.venv`) to simulate remote Internet clients. Conversely, the Reverse Proxy, Gateway, Cache, and AI Server run confined within their respective isolated Docker containers.

As shown in Figure 2.1, the four microservices (`reverse-proxy`, `ai-api-gateway`, `ai-application-server`, and `redis-cache`) are orchestrated and monitored in a coordinated manner, maintaining a controlled and compact memory and CPU footprint.

2.2 Networking Fundamentals and TCP Protocol Security

2.2.1 Port Mapping and Topological Decoupling

The infrastructure enforces a strict separation between externally exposed ports and internal microservice ports:

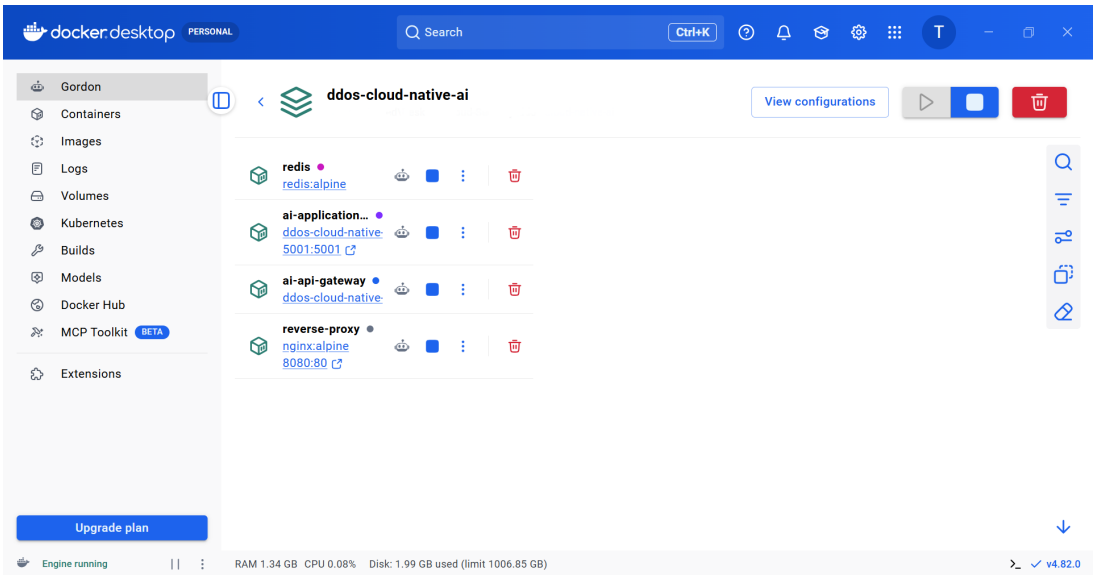


Figure 2.1: Overview of the 4 running containers inside Docker Desktop showing mapped ports and allocated resources.

- **Single Public Ingress Point**: Only host port `8080` is exposed externally, mapped to port `80` of the Nginx reverse proxy.
- **Unexposed Internal Services**: The API Gateway (port `8000`), the AI Inference Server (port `5001`), and the Redis database (port `6379`) are not directly reachable from the outside under normal operating conditions, preventing any unmediated access or scanning attempts.

2.2.2 Mitigating IP Spoofing via the TCP 3-Way Handshake

IP Spoofing (forging the source IP address in the IP packet header) is a common technique in attacks leveraging connectionless protocols (UDP Floods or DNS/NTP amplification). However, against Layer 7 application services built on the **TCP** transport protocol compliant with RFC 9293 [4] (such as the REST APIs of the AI service), direct IP spoofing is ineffective for delivering application-layer requests.

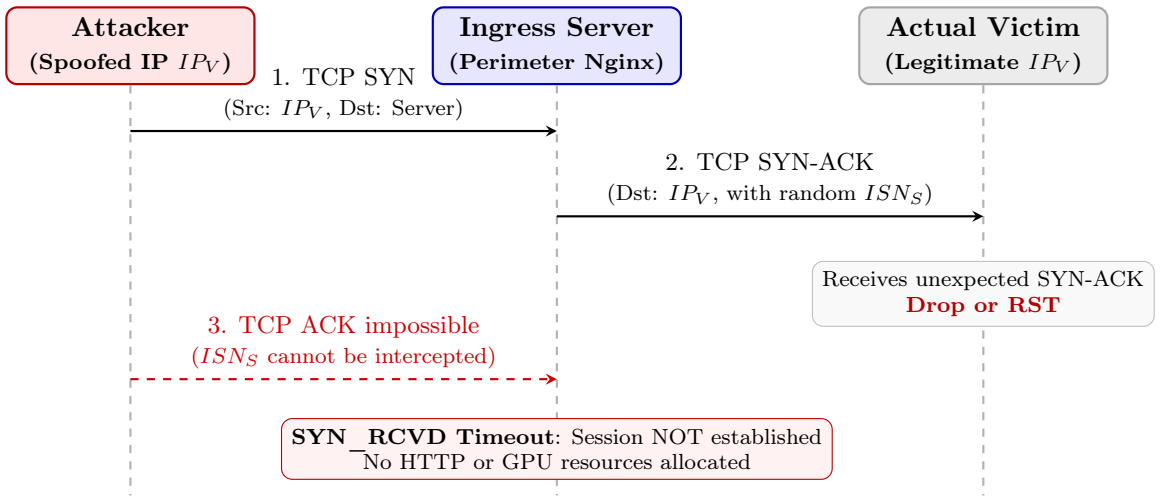


Figure 2.2: 3-actor TCP 3-Way Handshake flow: the attacker never receives the SYN-ACK packet containing ISN_S and cannot complete the HTTP connection.

As depicted in the sequence diagram in Figure 2.2:

1. The attacker sends a `SYN` packet to the server specifying a forged source IP address IP_V.
2. The server responds with a `SYN-ACK` packet routed to the legitimate owner of address IP_V. This packet contains the *Initial Sequence Number* (ISN_S) randomly generated by the server.
3. The attacker can neither receive nor predict the value of ISN_S. Consequently, they cannot send a valid final `ACK` packet to complete the 3-way handshake.

Because the TCP session never reaches the `ESTABLISHED` state, the server neither allocates application resources nor accepts HTTP POST requests. IP spoofing over TCP is therefore restricted to Layer 4 volumetric SYN Flood attacks, which are mitigated at the perimeter and by the Linux kernel before ever reaching the Gateway.

2.3 Multi-Tier Network Topology (VPC)

The network configuration adopts a two-tier model within a Virtual Private Cloud (VPC) provisioned via Docker Network:

1. **Public Subnet (`frontend_net` - DMZ)**: A bridge network accessible from the host where Nginx is deployed. It acts as the primary perimeter filter to absorb volumetric traffic spikes and handle secure channel termination over TLS 1.3 in compliance with RFC 8446 [5].
2. **Isolated Private Subnet (`backend_net` - `internal: true`)**: A network configured with the `internal: true` directive. This subnet has no outbound gateway to the Internet and completely isolates Redis and the Inference Server, making them accessible exclusively from the API Gateway.

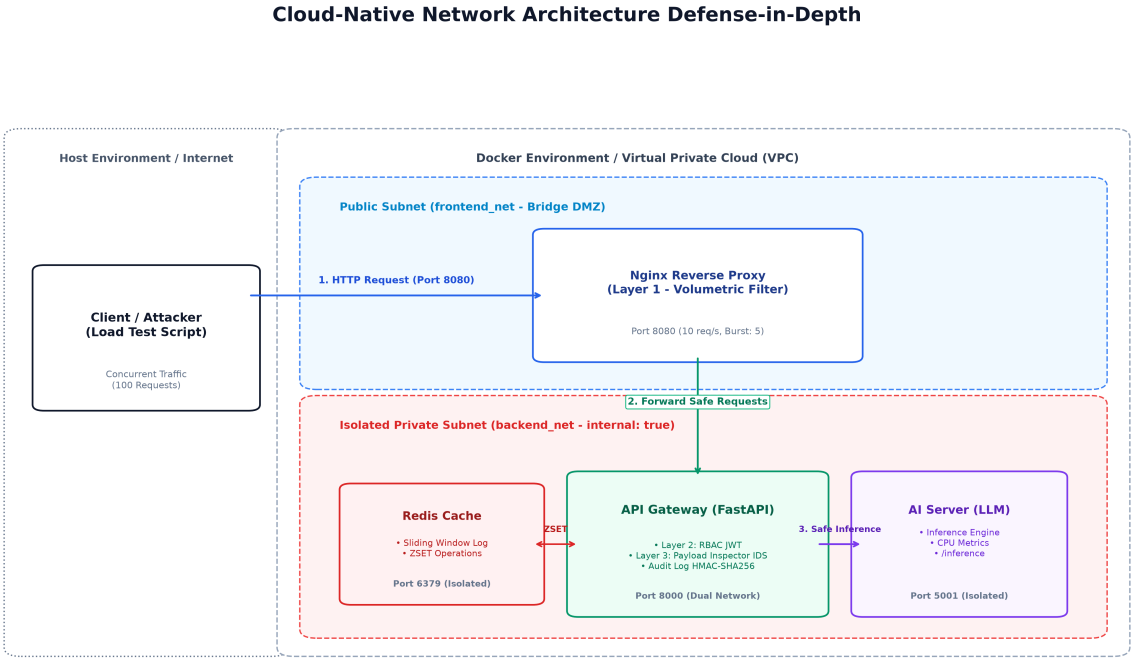


Figure 2.3: Logical topology of the Cloud-Native Defense-in-Depth network architecture segmented into a DMZ subnet and an internal private subnet.

Figure 2.3 illustrates the complete routing flow:

1. The external client sends an HTTP request to port `8080`.
2. Nginx enforces the initial volumetric check (Layer 1). If the request rate exceeds the threshold, it drops the request with an `HTTP 429` status code; otherwise, it forwards it to the Gateway over `frontend_net`.
3. The Gateway (dual-homed on both `frontend_net` and `backend_net`) performs JWT authentication, verifies user quotas in Redis via the Sliding Window algorithm (Layer 2), and inspects the prompt content using the application-layer IDS (Layer 3).
4. If all checks pass, the request is forwarded to the AI Server on port `5001` within the protected private network.

2.4 Microservice Composition and Docker Compose Configuration

The architecture comprises four distinct Docker images registered in the local daemon, as shown in Figure 2.4.

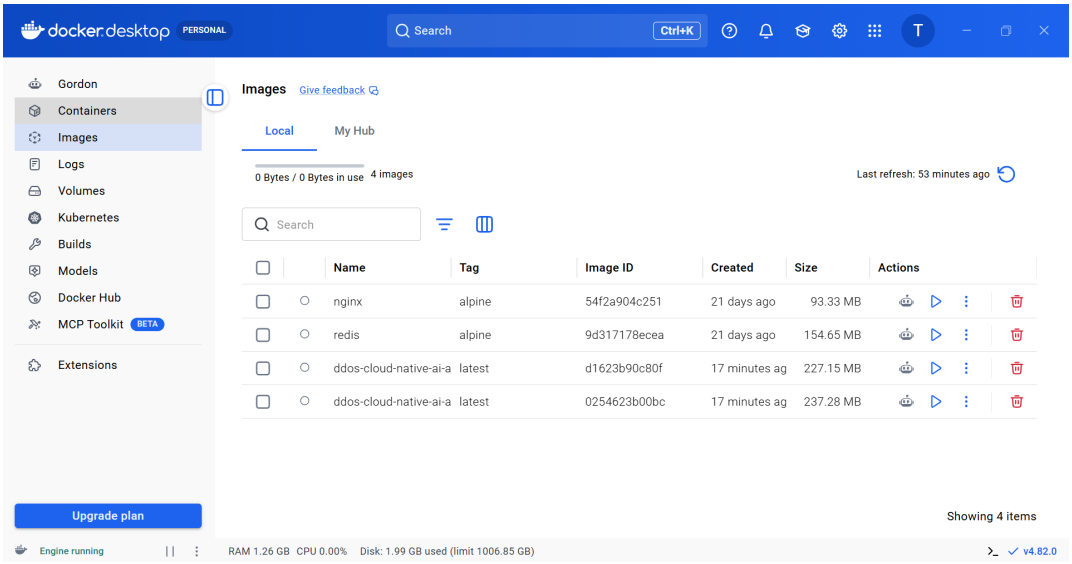


Figure 2.4: List of the 4 registered Docker images: third-party Alpine base images and custom-built microservice images.

The stack is divided into two image categories:

- **Pre-built Third-Party Images**:
  - `nginx:alpine` (approx. 93 MB): Official minimal image configured as a reverse proxy with the built-in rate limiting module enabled.
  - `redis:alpine` (approx. 154 MB): In-memory data store instance used to atomically record user request timestamps [8].
- **Custom Source-Built Images**:
  - `ai-api-gateway` (based on `python:3.11-slim`): Implements the FastAPI application [9] served by Uvicorn ASGI workers [10]. Within this microservice, **Uvicorn** handles non-blocking asynchronous multiplexing of incoming TCP connections (via *uvloop*), while **FastAPI** orchestrates the security module pipeline: cryptographic JWT validation, Redis sliding-window rate limiting, heuristic prompt inspection, and HMAC-SHA256 signed audit logging.

- `ai-application-server` (based on `python:3.11-slim`): A FastAPI and Uvicorn web service exposing the simulated inference endpoint—complete with parameterized computational load—and the CPU telemetry endpoint.

2.4.1 Orchestration Specification: docker-compose.yml

The complete orchestration of the four containers and the segregation of the two virtual networks are defined in the following configuration file:

```
version: '3.8'

networks:
  frontend_net:
    driver: bridge
  backend_net:
    driver: bridge
    internal: true  # Private subnet isolated from the Internet

services:
  redis:
    image: redis:alpine
    container_name: redis-cache
    networks:
      - backend_net
    restart: unless-stopped

  ai-application-server:
    build:
      context: ./ai-server
      dockerfile: Dockerfile
    container_name: ai-application-server
    ports:
      - "5001:5001"  # Exposed to simulate direct attack (Scenario A)
    networks:
      - frontend_net
      - backend_net
    restart: unless-stopped

  ai-api-gateway:
    build:
      context: ./gateway
      dockerfile: Dockerfile
    container_name: ai-api-gateway
    ports:
      - "8000:8000"  # Exposed for methodological isolation of Scenarios C & D (Nginx Bypass)
    env_file:
      - .env
    networks:
      - frontend_net
      - backend_net
    depends_on:
      - redis
```

```
      - ai-application-server
    restart: unless-stopped

  reverse-proxy:
    image: nginx:alpine
    container_name: nginx-reverse-proxy
    ports:
      - "8080:80"
    volumes:
      - ./reverse-proxy/nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - frontend_net
    depends_on:
      - ai-api-gateway
    restart: unless-stopped
```

Through this structure, the infrastructure ensures fault isolation, immediate reproducibility via a single startup command (`docker compose up -d -build`), and multi-layered protection against Denial of Service attacks.

Chapter 3

Implementation of the Three Defense-in-Depth Layers

This chapter presents the practical implementation of the three security layers comprising the Defense-in-Depth pipeline, analyzing the configuration code, control algorithms, and underlying data structures.

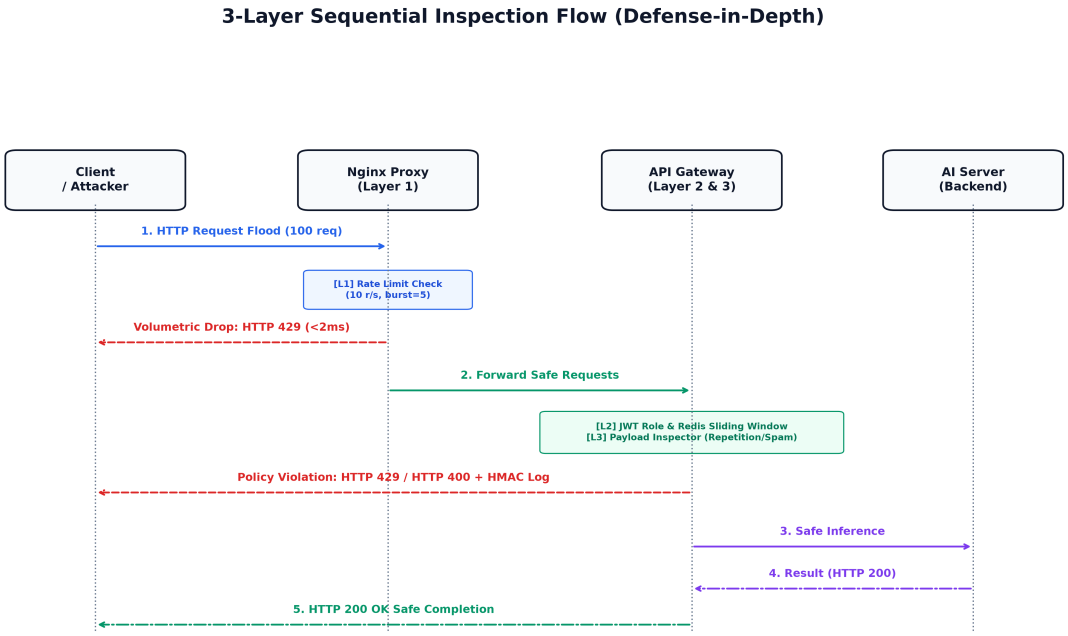


Figure 3.1: Multi-layer inspection sequence diagram: from immediate perimeter drop to semantic payload validation.

As shown in Figure 3.1, each request traverses a series of sequential checks. If any check fails, the request is immediately terminated at the current layer, preventing unnecessary computational resource allocation at downstream stages.

3.1 Layer 1: Perimeter Volumetric Filtering (Nginx)

The first line of defense is deployed on the Nginx reverse proxy located in the public subnet (DMZ). Its primary role is to absorb and filter L4/L7 volumetric traffic spikes before they reach the API Gateway.

In `reverse-proxy/nginx.conf`, the native `ngx_http_limit_req_module` is configured with the following directives:

```
http {
    # Shared memory zone definition indexed by client IP
    limit_req_zone $binary_remote_addr zone=api_limit_zone:10m rate=10r/s;

    server {
        listen 80;

        location /api/v1/ {
            # Rate limit enforcement with instantaneous burst and nodelay
            limit_req zone=api_limit_zone burst=5 nodelay;
            limit_req_status 429;

            proxy_pass http://ai-api-gateway:8000;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
```

3.1.1 Technical Breakdown of Nginx Parameters

- `$binary_remote_addr`: Stores the client IP address in binary format (4 bytes for IPv4 instead of up to 15 bytes for a string representation), significantly reducing memory footprint.
- `zone=api_limit_zone:10m`: Allocates a 10-megabyte shared memory zone within the Nginx master process, sufficient to track the state of approximately 160,000 concurrent IP addresses.
- `rate=10r/s`: Caps the maximum sustained request rate at 10 requests per second per individual IP.
- `burst=5 nodelay`: Accommodates up to 5 instantaneous requests exceeding the nominal rate without introducing artificial queuing delays (*nodelay*). Any subsequent requests exceeding the burst capacity are immediately dropped with an `HTTP 429 Too Many Requests` status code, yielding an average response time under 2 milliseconds.

3.2 Layer 2: Role-Based Access Control (RBAC) and Atomic Rate Limiting

The second defense layer operates within the API Gateway built on the asynchronous FastAPI framework [9] (served by the Uvicorn ASGI server [10]) and combines cryptographic authentication with dynamic usage quota enforcement.

3.2.1 JSON Web Token (JWT) Authentication Model

Client identity and privileges are verified using JWT tokens compliant with RFC 7519 [6]. The token payload encapsulates the user identifier claim (`sub`) and the assigned role (`role`), cryptographically signed using the `HS256` algorithm.

Based on the role extracted from the token, the Gateway enforces the policies detailed in Table 3.1.

Role	Quota Limit	Time Window	Application Policy
Admin	Unlimited	-	Bypasses the volumetric rate limiter. Remains subject to IDS inspection and audit logging.
Employee	60 requests	60 seconds	Standard profile for authenticated enterprise users.
Guest	5 requests	60 seconds	Restrictive profile for anonymous or trial users.

Table 3.1: Traffic profile and quota matrix for RBAC access control.

3.2.2 Mitigating Race Conditions (TOCTOU) and Shared State via Redis

Adopting the Redis in-memory database [8] addresses two fundamental architectural requirements of cloud-native applications:

1. **Shared State for Horizontal Scalability**: In a distributed microservice infrastructure, incoming traffic is load-balanced across multiple concurrent API Gateway replicas. Maintaining rate-limiting counters in the local memory of individual processes (e.g., in-memory Python dictionaries) would allow a malicious client to multiply their effective quota by distributing requests across different instances. Redis acts as a centralized, synchronized state store accessible to all Gateway replicas.

2. **Eliminating Race Conditions (TOCTOU) via Lua Scripting**: In high-concurrency distributed systems, an unsynchronized sequence of operations (reading the counter → comparing against the threshold → incrementing the value) exposes the system to **Time-of-Check to Time-of-Use (TOCTOU)** vulnerabilities. Two or more concurrent requests arriving within the same millisecond would read the same stale counter value, bypassing the maximum quota. By delegating the entire control logic to a Lua script executed within Redis's single-threaded engine [8], check-and-update operations are executed strictly atomically.

The monolithic Lua script embedded in the Gateway implements the **Sliding Window Log** algorithm:

```
local key = KEYS[1]
local clear_before = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local current_time = ARGV[3]
local member_id = ARGV[4]
local window_seconds = tonumber(ARGV[5])

-- 1. Atomic eviction of records older than the sliding time window
redis.call('ZREMRANGEBYSCORE', key, 0, clear_before)

-- 2. Count active requests within the current window
local current_requests = redis.call('ZCARD', key)

-- 3. Compare against the maximum allowed quota
if current_requests >= limit then
    return 0 -- Quota exceeded: request rejected (HTTP 429)
else
    -- 4. Atomic timestamp insertion and TTL expiration renewal
    redis.call('ZADD', key, current_time, member_id)
    redis.call('EXPIRE', key, window_seconds)
```

```
    return 1 -- Request authorized
end
```

Using a Sorted Set (`ZSET`) data structure indexed by a timestamp score records the exact arrival instant of each request and fundamentally eliminates the boundary "double burst" edge case inherent to Fixed Window counters, where an attacker can concentrate twice the allowed requests across a minute boundary.

3.2.3 Cryptographic Audit Logging with HMAC-SHA256 Signatures

All request routing, acceptance, and blocking decisions are serialized by the `audit_logger.py` module into structured JSON format. To ensure log files cannot be retroactively altered by an attacker who compromises the host (*tamper-evidence*), each log entry embeds a cryptographic signature computed via the `HMAC-SHA256` algorithm compliant with RFC 2104 [7] using a dedicated secret key:

```
{
  "timestamp": "2026-08-20T12:00:15.842100+00:00",
  "client_ip": "172.19.0.1",
  "role": "guest",
  "endpoint": "/api/v1/inference",
  "status_code": 429,
  "decision": "BLOCKED",
  "detail": "Rate limit exceeded (Max: 5/60s)",
  "risk_score": 0.0,
  "signature": "hmac_sha256_7b8a1c90e3d4f5..."
}
```

3.3 Layer 3: Application IDS and Payload Inspector

The third protection layer analyzes the syntactic and semantic structure of the prompt received within the HTTP request body. The `payload_inspector.py` module intercepts computational exhaustion attempts by computing a cumulative risk score function $R(P) \in [0, 1]$:

$$R(P) = w_l \cdot S_{\text{length}}(P) + w_r \cdot S_{\text{rep}}(P) + w_d \cdot S_{\text{dup}}(P) \tag{3.1}$$

with empirical weights set to $w_l = 0.35$, $w_r = 0.35$, $w_d = 0.30$, subject to the constraint $\sum w_i = 1.0$.

The individual partial risk indices are defined as follows:

1. **Length Check (S_{length})**: Measures proximity to the maximum allowed character threshold (500 characters):
$$S_{\text{length}}(P) = \min\left(1.0,\ \frac{\text{len}(P)}{500}\right) \tag{3.2}$$

2. **Lexical Repetition Ratio (S_{rep})**: Detects strings containing repeated identical words designed to trigger compute loops:
$$S_{\text{rep}}(P) = \begin{cases} 1.0 & \text{if } \left(1.0 - \dfrac{|W_{\text{unique}}|}{|W_{\text{total}}|}\right) > 0.3 \quad (\text{with } |W_{\text{total}}| \geq 5) \\ 0.0 & \text{otherwise} \end{cases} \tag{3.3}$$

3. **Temporal Prompt Duplication ($S_{\mathbf{dup}}$)**: Computes the MD5 hash of the prompt and tracks how many times it has been submitted within the last 15 seconds:

$$S_{\text{dup}}(P) = \begin{cases} 1.0 & \text{if } \text{count}_{15s}(\text{MD5}(P)) \geq 3 \\ 0.0 & \text{otherwise} \end{cases} \tag{3.4}$$

If the composite score exceeds the risk threshold ($R(P) > 0.7$), the request is immediately aborted with an `HTTP 400 Bad Request` status code, preventing GPU cycle consumption on the backend inference server.

Chapter 4

Experimental Validation and Results Analysis

This chapter presents the results obtained from the experimental testing campaign conducted to evaluate the effectiveness of the implemented protections against volumetric and semantic DoS attacks.

4.1 Measurement Methodology and Testbed Setup

Testing was performed by simulating concurrent traffic bursts generated by client scripts executing within the host virtual environment (`load-test/attack_simulation.py` and `load-test/protected_run.py`). To ensure statistical robustness, $N = 10$ independent test runs of 100 concurrent requests each were executed across all 4 operational scenarios.

4.1.1 Ingress Points and Methodological Layer Decoupling

The experimental setup employs three distinct ingress points to isolate and independently verify the effectiveness of each defensive layer:

- **Scenario A (Unprotected Baseline)**: The load generator dispatches 100 requests directly to port `5001` of the AI Server, simulating the complete absence of security controls.
- **Scenario B (Perimeter Guest Profile)**: Requests are directed to port `8080` of the Nginx reverse proxy. This scenario evaluates the combined action of Layer 1 IP-based rate limiting (`rate=10r/s burst=5`) and the baseline quota for anonymous users (5 req/min) enforced by the Gateway.
- **Scenarios C (Employee) and D (Admin) via Perimeter Bypass**: In tests executed from a single laboratory host node, all 100 generated requests share the same source IP address (`127.0.0.1`). If these bursts traversed Nginx, the Layer 1 filter (which is agnostic to application-layer user identity) would drop nearly all of them at the perimeter before they could reach the Gateway. To rigorously evaluate the Layer 2 RBAC policies in isolation (the enterprise quota of 60 req/min for employees and unrestricted access for administrators), the test scripts route traffic directly to Gateway port `8000` (`-bypass-nginx`), emulating access originating from a corporate VPN or a trusted internal subnet.

The observed sampling parameters and statistical variability metrics are:

- **Burst latency standard deviation**: $\sigma \leq \pm 0.14\,\text{s}$.
- **CPU peak standard deviation**: $\sigma \leq \pm 2.1\%$.

- **Metrics sampling interval**: active hardware polling every 300 ms throughout the burst window.

4.2 Computational Load Modeling and Telemetry

In network infrastructure security research, the backend component is modeled as an asymmetric computational function characterized by an $O(N)$ cost profile relative to available resources. The effectiveness of the defensive chain (Nginx L4/L7 filters, JWT authentication, Redis rate-limiting algorithms, and payload inspection) operates upstream and is architecturally decoupled from the specific tensor engine deployed.

Adopting a parameterized computational model satisfies strict methodological and engineering criteria:

- **Focus on the Protection Infrastructure**: The primary validation objective is measuring the perimeter's ability to intercept and drop abusive traffic before it engages backend compute capacity, making the specific model weight architecture orthogonal to the evaluation.
- **Scientific Portability and Reproducibility**: It eliminates strict hardware dependencies on dedicated enterprise-grade physical GPUs (requiring 16–24 GB of VRAM), enabling immediate, deterministic reproduction of the entire containerized cluster on any standard host node.
- **Experimental Determinism**: It removes measurement noise introduced by thermal throttling, dynamic CUDA video memory allocations, and system jitter, ensuring that cross-scenario comparisons occur under strictly identical operating conditions.

Inference processing time is modeled via an asynchronous function proportional to the prompt character length L:

$$T_{\text{inference}}(L) = \min\left(5.0,\ L \times 0.005\right) \quad \text{[seconds]} \tag{4.1}$$

The theoretical CPU saturation load during the burst window ($W = 3.0\,\text{s}$) models compute core contention across active concurrent requests $N_{\text{active}}(t)$:

$$\text{CPU}_{\text{peak}}(t) = \min\left(98.5\%,\ \text{CPU}_{\text{base}} + N_{\text{active}}(t) \times 10.0\%\right) \tag{4.2}$$

where $\text{CPU}_{\text{base}} = 2.5\%$ represents the baseline idle consumption of the process and the 98.5% cap reflects the minimum operational headroom reserved for the kernel scheduler.

The `/metrics` endpoint concurrently exposes two distinct telemetry metrics:

1. `cpu_peak_during_burst`: The theoretical maximum peak computed across active threads during the critical contention window.
2. `cpu_current`: The actual real-time consumption measured on the container via `psutil.cpu_percent()`, which returns to 0.0% upon completion of the request burst.

Furthermore, RAM monitoring via `psutil.virtual_memory()` reveals a stable memory footprint of approximately 11.7%. This finding confirms the strictly **CPU-bound** (computational saturation) nature of semantic DoS attacks: compact JSON payloads (occupying only a few kilobytes in volatile memory) impose prolonged tensor processing times on the backend without triggering memory leaks or address space exhaustion.

Measured Metric	**Scenario A** (Direct Attack)	**Scenario B** (Protected Guest)	**Scenario C** (Employee)	**Scenario D** (Admin)
Total Sent Requests	100	100	100	100
Accepted Requests (`HTTP 200`)	100	5	60	100
Rejected Requests (`HTTP 429/400`)	0	95	40	0
Total Burst Duration	4.88 s	3.52 s	4.79 s	5.86 s
Mean Perceived Latency	3.93 s	0.75 s	1.64 s	3.89 s
Backend Peak CPU Load	98.5%	10.0%	60.0%	98.5%
Post-Test Idle CPU	0.8%	0.0%	0.0%	0.0%

Table 4.1: Performance and mitigation effectiveness comparison between Scenario A (baseline) and protected Scenarios B, C, and D ($N = 10$).

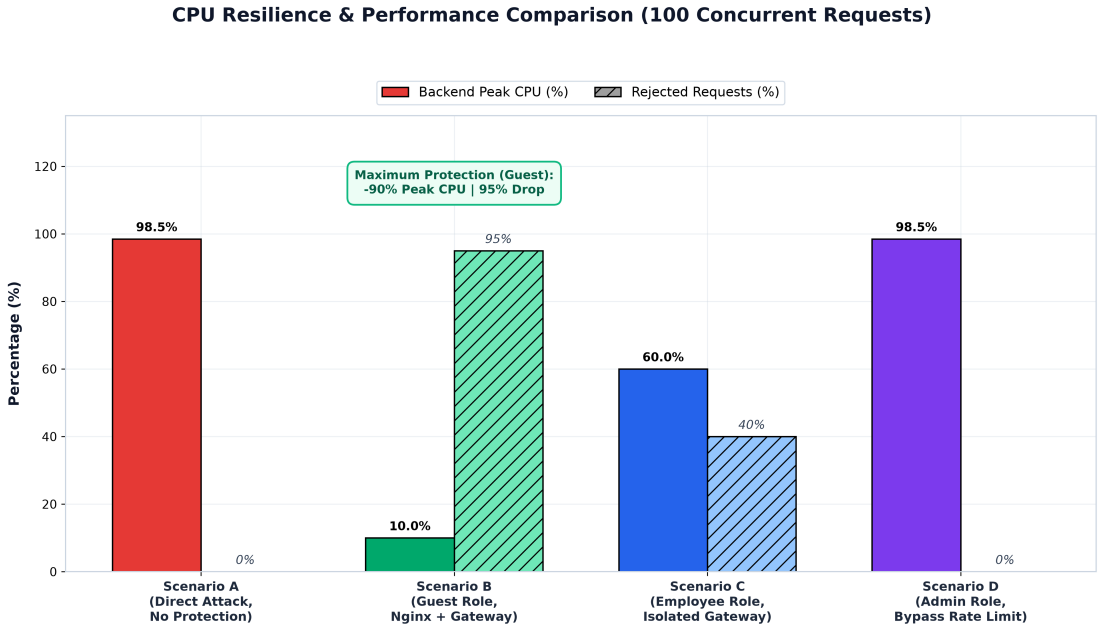


Figure 4.1: Comparison of peak CPU utilization and rejected request percentage across the 4 load scenarios.

4.3 Comparative Analysis of Experimental Scenarios

The mean results recorded across the $N = 10$ test sessions are summarized in Table 4.1.

As highlighted in Figure 4.1:

- In **Scenario A (Direct Attack)**, the absence of perimeter controls results in the simultaneous processing of all 100 requests, saturating the AI server CPU at **98.5%** and rendering the service unavailable to legitimate users.
- In **Scenario B (Guest Role)**, the combined enforcement of Nginx and Gateway rate limiting intercepts and drops **95%** of abusive requests. Only the 5 authorized requests reach the AI Server, capping CPU load at **10.0%** (an approximate 90% reduction in computational load).
- In **Scenario C (Employee Role)**, the Redis sliding-window algorithm throttles requests within the enterprise quota of 60 per minute, rejecting the 40 excess requests.
- In **Scenario D (Admin Role)**, the volumetric rate limiter is bypassed as dictated by service policy, allowing all 100 requests to complete.

4.4 Time Efficiency and Drop Latency Analysis

A critical factor in DoS mitigation is the speed at which unauthorized requests are dropped at the perimeter, freeing network and compute resources.

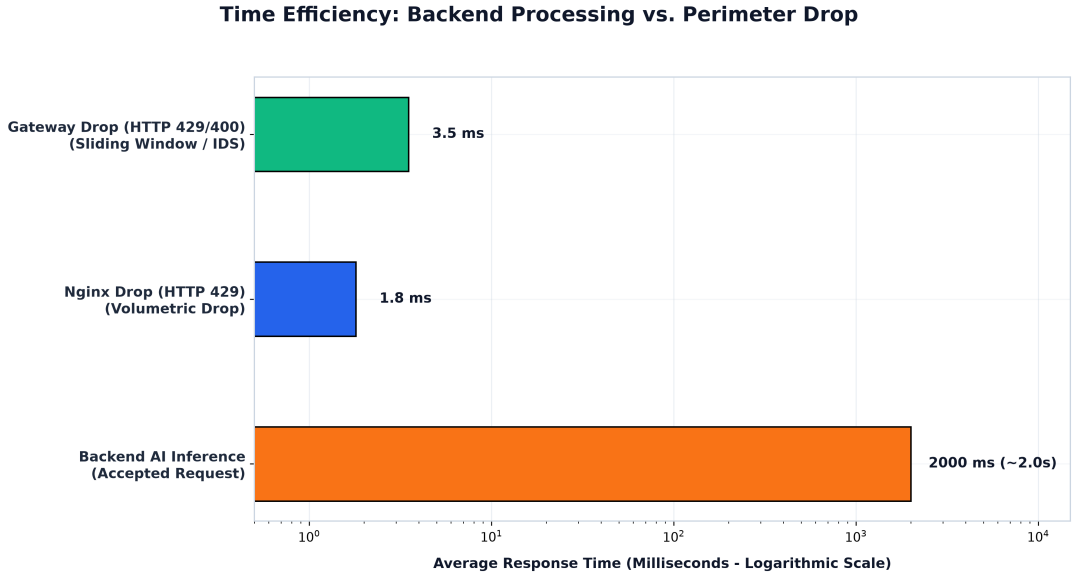


Figure 4.2: Comparison between mean AI inference processing time and immediate perimeter drop latency (logarithmic scale).

The chart in Figure 4.2 illustrates the stark temporal asymmetry between backend inference processing and perimeter rejection:

- A legitimate inference completed by the AI Server requires an average of approximately **2000 milliseconds** (2.0 s).
- The Nginx perimeter filter (Layer 1) drops excess requests in just **1.8 milliseconds**.
- The FastAPI Gateway—performing JWT claim inspection and Redis quota verification (Layers 2 and 3)—executes rejections in approximately **3.5 milliseconds**.

Through this temporal decoupling, the system eliminates malicious traffic at a computational cost more than three orders of magnitude lower than that required to process an inference, effectively neutralizing the asymmetry of the attack.

Chapter 5

Design Trade-offs, Limitations, and Future Work

This concluding chapter examines the primary engineering trade-offs made during the architectural design, the current limitations of the prototype, and potential extensions for high-reliability enterprise environments.

5.1 Critical Analysis of Architectural Trade-offs

Implementing the system required balancing several trade-offs across operational complexity, performance, and security robustness:

- **Redis vs. Memcached for Rate Limiting**: Memcached guarantees minimal latency on simple key-value tuples, but lacks support for complex data structures and server-side atomic script execution. Adopting Redis [8] proved essential to leverage *Sorted Sets* (`ZSET`) and transactional Lua scripts, which are indispensable for implementing the Sliding Window algorithm without introducing race condition vulnerabilities.
- **Sliding Window Log vs. Token Bucket / Fixed Window**: Fixed Window counters suffer from the boundary double-burst problem across adjacent time intervals, whereas Token Bucket algorithms permit initial bursts that can temporarily saturate the AI backend. The Redis-backed Sliding Window eliminates these discontinuities, enforcing continuous, uniform traffic shaping.
- **Docker Compose vs. Kubernetes**: For this Proof-of-Concept, Docker Compose was favored over a full Kubernetes cluster to minimize network virtualization overhead (CNI overlays) and enable immediate, deterministic reproduction of all test suites on a single host node.

5.2 Stated Study Limitations

To ensure methodological transparency, the following limitations of the implemented prototype are noted:

1. **Symmetric JWT Cryptography (`HS256` vs. `RS256/ES256`)**: Employing the symmetric `HS256` algorithm requires sharing the same secret key (`JWT_SECRET`) between the token issuer and the Gateway. In production environments, migrating to asymmetric key pairs (`RS256` or `ES256`) is recommended, allowing the Identity Provider to sign tokens with a private key while the Gateway validates them using a read-only public key.

2. **Audit Log Non-Repudiation**: The `HMAC-SHA256` signature guarantees that log records have not been modified (*tamper-evidence*), but does not provide strict cryptographic non-repudiation because the signing key is symmetric. In high-assurance forensic contexts, asymmetric signatures backed by Hardware Security Modules (HSMs) or Cloud KMS are preferable.

3. **IDS Resilience Against High-Vocabulary Dictionary Attacks**: Heuristic inspection based on the lexical repetition ratio (`repetition_ratio`) effectively intercepts duplicate-word spam attempts, but is not designed to detect nonsensical text generated from randomized dictionary words (where lexical repetition approaches zero). Defending against such patterns requires specialized semantic guardrails based on vector embeddings or dedicated classifiers (e.g., Llama Guard).

4. **CPU Load Simulation vs. Physical GPUs**: Parameterized load modeling faithfully reproduces the logical queuing bottlenecks of inference pipelines, but does not replicate the thermal dynamics and PCIe bus saturation inherent to enterprise-grade physical GPUs.

5.3 Future Work

The implemented architecture is designed to accommodate the following enterprise extensions:

- **SIEM Integration**: Streaming structured, cryptographically signed audit logs to centralized event correlation platforms (OpenSearch, Splunk, or the Elastic Stack) for real-time security incident monitoring.
- **Machine Learning Anomaly Detection**: Embedding lightweight classifiers within the Gateway for predictive analysis and detection of prompt injections or complex volumetric-semantic prompts.
- **High Availability (HA)**: Migrating the Redis cache to redundant clusters managed via Redis Sentinel and deploying multiple Nginx ingress nodes load-balanced via DNS or BGP (Anycast) to eliminate any Single Point of Failure (SPOF).

5.4 Conclusions

This work demonstrated how adopting a **Defense-in-Depth** paradigm structured across three sequential tiers—perimeter volumetric filtering, granular RBAC authentication with atomic Redis rate limiting, and heuristic prompt inspection—effectively protects cloud-native AI inference services.

Experimental results confirm a **95%** reduction in abusive perimeter requests, capping backend computational load at **10.0%** (compared to 98.5% observed without defenses), achieving perimeter drop latencies under 2 milliseconds, and preserving full operational continuity for authorized users.

Bibliography

[1] National Institute of Standards and Technology (NIST), *Zero Trust Architecture*, NIST Special Publication 800-207, August 2020. https://doi.org/10.6028/NIST.SP.800-207

[2] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications (v1.1)*, Focus on *LLM04: Model Denial of Service*, 2023. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[3] OWASP Foundation, *OWASP API Security Top 10*, Focus on *API4:2023 Unrestricted Resource Consumption*, 2023. https://owasp.org/www-project-api-security/

[4] W. Eddy, *Transmission Control Protocol (TCP) Specification*, RFC 9293, IETF Internet Standard, August 2022. https://datatracker.ietf.org/doc/html/rfc9293

[5] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF Proposed Standard, August 2018. https://datatracker.ietf.org/doc/html/rfc8446

[6] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF Proposed Standard, May 2015. https://datatracker.ietf.org/doc/html/rfc7519

[7] H. Krawczyk, M. Bellare, and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF Informational, February 1997. https://datatracker.ietf.org/doc/html/rfc2104

[8] Redis Ltd., *Redis Documentation: Sorted Sets (ZSET), Lua Scripting and Atomic Operations*, 2024. https://redis.io/docs/

[9] S. Ramírez, *FastAPI Framework Documentation: High Performance Asynchronous Web APIs with Python*, 2024. https://fastapi.tiangolo.com/

[10] Encode OSS, *Uvicorn: An ASGI Web Server Implementation for Python*, 2024. https://www.uvicorn.org/